

EXP 9: Travelling Salesman Problem (TSP)

Aim

To implement the **Travelling Salesman Problem (TSP)** using a brute-force approach in Python to find the shortest possible route that visits each city exactly once and returns to the starting city.

Algorithm

1. Define the distance matrix between all cities.
2. Select the starting city.
3. Generate all possible permutations of the remaining cities.
4. Calculate the total travel cost for each possible route.
5. Compare all route costs.
6. Store the route with the minimum cost.
7. Display the shortest path and its minimum cost.

Code

```
from itertools import permutations

graph = [
    [0, 10, 15, 20],
    [10, 0, 35, 25],
    [15, 35, 0, 30],
    [20, 25, 30, 0]
]

n = len(graph)
start = 0
cities = list(range(n))
cities.remove(start)
min_cost = float('inf')
best_path = None
for perm in permutations(cities):
    cost = 0
```

```
current = start
```

```
for city in perm:
```

```
    cost += graph[current][city]
```

```
    current = city
```

```
cost += graph[current][start]
```

```
if cost < min_cost:
```

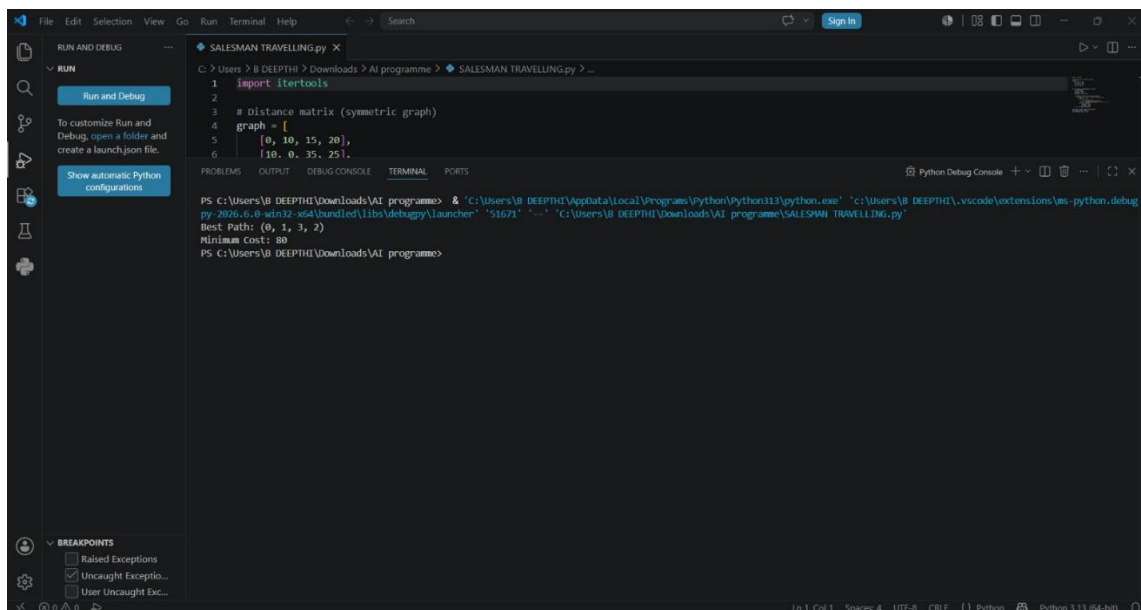
```
    min_cost = cost
```

```
    best_path = (start,) + perm + (start,)
```

```
print("Shortest Path:", best_path)
```

```
print("Minimum Cost:", min_cost)
```

Output



```
SALESMAN TRAVELLING.py X
C:\Users\B DEEPTI\Downloads\AI programme> SALESMAN TRAVELLING.py
1 import itertools
2
3 # Distance matrix (symmetric graph)
4 graph = [
5     [0, 10, 15, 20],
6     [10, 0, 35, 25],
7 ]
8
9 # Iterate through all permutations of cities (1, 2, 3)
10 for perm in itertools.permutations([1, 2, 3]):
11     # Start at city 0, visit cities in perm, and return to 0
12     current = 0
13     cost = 0
14     for city in perm:
15         cost += graph[current][city]
16         current = city
17     cost += graph[current][0]
18
19     if cost < min_cost:
20         min_cost = cost
21         best_path = (0,) + perm + (0,)
22
23 # Print the results
24 print("Shortest Path:", best_path)
25 print("Minimum Cost:", min_cost)
```

Python Debug Console

```
PS C:\Users\B DEEPTI\Downloads\AI programme> & "c:\Users\B DEEPTI\AppData\Local\Programs\Python\python313\python.exe" "c:\Users\B DEEPTI\.vscode\extensions\ms-python.debugpy-2024.6.8-win32-x64\libs\debugpy\launcher" "51671" -c "c:\Users\B DEEPTI\Downloads\AI programme\SALESMAN TRAVELLING.py"
Best Path: (0, 1, 3, 2)
Minimum cost: 80
PS C:\Users\B DEEPTI\Downloads\AI programme>
```

Result

The Travelling Salesman Problem was successfully implemented using the brute-force approach. The algorithm identified the shortest possible route with a minimum travel cost of **80**.

EXP 10: A* Algorithm

Aim

To implement the A* search algorithm in Python to find the shortest path between a start node and a goal node using heuristic values.

Algorithm

1. Create the graph with edge costs.
2. Define heuristic values for each node.
3. Initialize the open list with the start node.
4. Select the node having the lowest $f(n)=g(n)+h(n)$.
5. Expand the selected node.
6. Update the cost of neighboring nodes.
7. Repeat until the goal node is reached.
8. Display the shortest path and total cost.

Code

```
from queue import PriorityQueue
```

```
graph = {  
    'A': [('B', 1), ('C', 4)],  
    'B': [('D', 2), ('E', 5)],  
    'C': [('F', 3)],  
    'D': [('G', 4)],  
    'E': [('G', 2)],  
    'F': [('G', 5)],  
    'G': []  
}
```

```
heuristic = {  
    'A': 7,  
    'B': 6,
```

'C': 4,
'D': 2,
'E': 1,
'F': 3,
'G': 0

```
        current = came_from[current]
        path = [goal]
        while current in came_from:
    }
def astar(start, goal):
    pq = PriorityQueue()
    pq.put((0, start))
    came_from = {}
    g_cost = {start: 0}
    while not pq.empty():

        _, current = pq.get()
        if current == goal:

            path.append(current)
            return path[::-1], g_cost[goal]
        for neighbor, cost in graph[current]:
            new_cost = g_cost[current] + cost

            if neighbor not in g_cost or new_cost < g_cost[neighbor]:
                g_cost[neighbor] = new_cost
                priority = new_cost + heuristic[neighbor]
                pq.put((priority, neighbor))
```

```

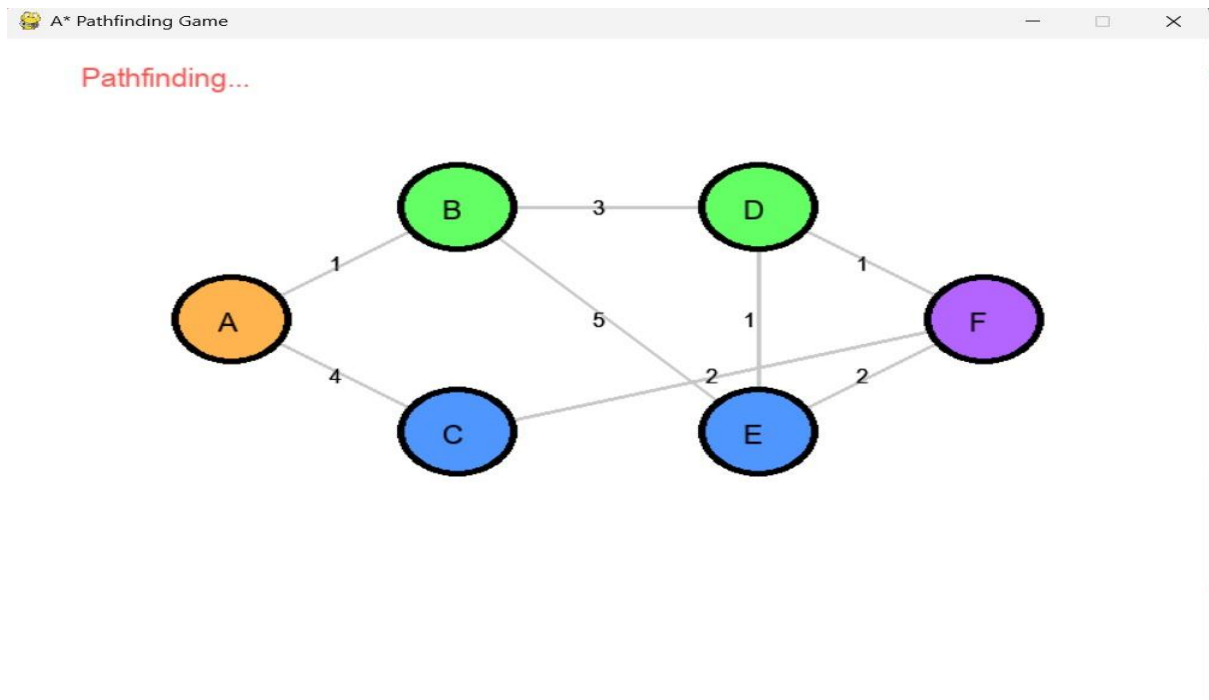
        came_from[neighbor] = current

    return None

path, cost = astar('A', 'G')
print("Shortest Path:", " -> ".join(path))
print("Total Cost:", cost)

```

Sample Output



Result

The A* algorithm was successfully implemented. It efficiently found the shortest path from the start node to the goal node by combining the actual path cost and heuristic estimate, resulting in the path **A → B → D → G** with a total cost of **7**.